

UNIT IV: Multithreading: introduction, thread life cycle, creation of threads, thread priorities, thread synchronization, communication between threads.

Multithreading Introduction:

1. Multithreading is a process of executing multiple tasks simultaneously where each task is a separate independent part of the same program is known as Multithreading.
2. Sharing of resources (cpu, i/o, memory) among multiple threads of the same program
3. CPU switches multiple threads of the same processes
4. Multiple threads may share a common address space
5. Light weight process
6. Context switching is low cost
7. Inter thread communication cost is low
8. Under control of OS and java

Advantages:

1. You **can perform many operations together, so it saves time.**
2. Threads are **independent**, so it doesn't affect other threads if an exception occurs in a single thread.

Thread Life Cycle (States):

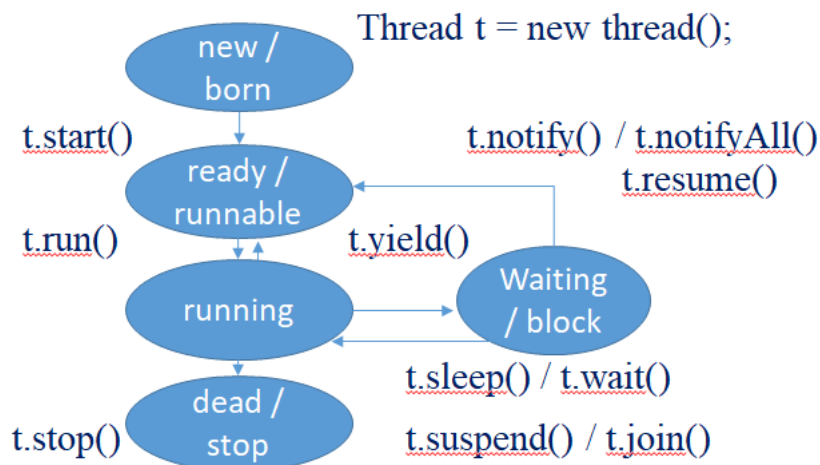
In Java, a thread always exists in any one of the following states. These states are:

1. New / Born
2. Ready / Runnable
3. Running
4. Waiting / Blocked
5. Stop / Dead

Explanation of Different Thread States:

- ✓ **New:** Whenever a thread created that will be in new state i.e., after creation of obj it will enter into new state.
- ✓ **Runnable:** If the thread is ready for execution then it enters into runnable state
- ✓ **Running:** If the processor (CPU) is available then it enters into running state
- ✓ **Waiting / Blocked:** If the thread requires any interruption like i/o occurs then it enters into waiting state. After getting the required thing it may enter either runnable (if the processor is busy) / running state (if the processor is available)
- ✓ **Stop:** After completion of thread execution it enters into stop state i.e., termination

The following diagram shows the different states involved in the life cycle of a thread.



Creation of Threads / Implementation of Thread:

- ✓ There are two ways to create a thread:
 1. By extending Thread class (Inheritance)
 2. By implementing Runnable interface. (Interface)
- ✓ The derived class can access the properties of thread class only
- ✓ The execution starts by using the start() method and it implicitly calls the run() method
- ✓ We need to write the actual code inside the run() method and it overrides the run() of thread class
- ✓ In interface, we can create object for the Thread class & pass the object of runnable interface as an argument for Thread class
- ✓ start() method can be called through Thread class object and it implicitly called run() method

Example - 1: Java Thread Example by extending Thread class

```

class MyThread extends Thread{
    public void run(){
        for(int i = 0;i < 10;i++)
        {
            System.out.println("Child Thread");
        }
    }
}

public class ThreadDemo1 {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start();
        for(int i = 0;i < 10;i++)
        {
            System.out.println("Main Thread");
        }
    }
}

```

```

    }
}
}

```

Output:

```

Main Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Main Thread
Main Thread
Main Thread
Main Thread

```

Example - 2: Java Thread Example by implementing Runnable interface

```

class MyRunnable implements Runnable{
    public void run(){
        for(int i = 0;i < 10;i++)
        {
            System.out.println("Child Thread");
        }
    }
}

public class ThreadDemo2 {
    public static void main(String[] args) {
        MyRunnable ob = new MyRunnable();
        Thread t1 = new Thread(ob);
        t1.start();
        for(int i = 0;i < 10;i++)
        {
            System.out.println("Main Thread");
        }
    }
}

```

Output

```

Main Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Child Thread
Main Thread
Main Thread
Main Thread
Main Thread

```

Methods of Thread class:

✓ Thread methods are available in java.lang package

1.start(): Thread execution will start from the start() method otherwise the thread will not start

Whenever a start() method is called then thread scheduler will check if cpu is free or not

2.run(): It is called implicitly by start() method when the cpu is free i.e., no need to call directly

3.sleep(ms): Thread will be suspended for given interval of time in milliseconds and thread execution will be resumed after the given time expires.

we should write in try and catch block & it is a static method

4.join(): Wait the other thread until currently executed thread is terminated & used in multithreading.

we should write in try and catch block & it is a static method (no need to create an object)

5.getId(): It returns the Identification number of a particular thread

6.getName(): It returns the name of a particular thread and the name starts from Thread-0 like array

7.setName(string): Thread name will be replaced with the given string

8.getPriority(): Every Thread has their own priority and it ranges from 1 – 10 and by default the priority of every thread is 5 (min priority - 1, normal priority - 5, max priority – 10)

9.setPriority(integer): We can change / set the priority values from 1 - 10

10.isAlive(): It returns the status of thread in the form of true (if thread is still running)/ false (if thread completes its execution)

11.yield(): It stops execution of currently running thread and gives control to some other thread with equal or greater priority

12.wait(integer): A thread goes into sleeping state / waiting state until some other thread calls notify()

13.notify(integer): It wakes up a thread that called wait() on same object

14.stop(): It stops execution of currently running thread

Example: Methods of Thread class

class MyThread **extends** Thread

```
{  
    public void run()  
    {  
        for(int i=0;i<=5;i++)  
        {  
            try  
            {  
                Thread.sleep(1000);  
            }  
        }  
    }  
}
```

```

    }
    catch(Exception e)
    {
        System.out.println(e);
    }
    System.out.println("Values of i="+i);
}
}
}
class ThreadDemo{
    public static void main(String args[]){
        MyThread t1=new MyThread ();
        MyThread t2=new MyThread ();
        System.out.println("ID="+t1.getId());
        System.out.println("Name="+t1.getName());
        t1.setName("Java");
        System.out.println("After changing, Name="+t1.getName());
        System.out.println("Priority of Thread="+t1.getPriority());
        t1.setPriority(10);
        System.out.println("After change,Priority of Thread="+t1.getPriority());
        t1.start();
        System.out.println("Thread Status="+t1.isAlive());
        try
        {
            t1.join();
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
        System.out.println("Thread Status="+t1.isAlive());
        t2.start();
    }
}

```

Output:

```

ID=20
Name=Thread-0
After changing, Name=Java
Priority of Thread=5
After change,Priority of Thread=10
Thread Status=true
Values of i=0
Values of i=1
Values of i=2
Values of i=3

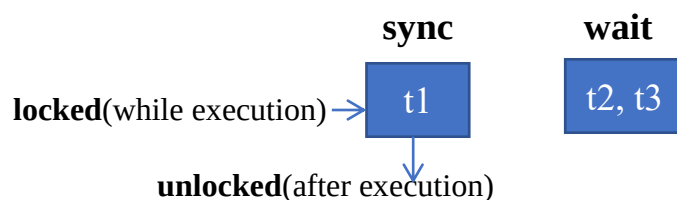
```

Values of i=4
 Values of i=5
 Thread Status=false
 Values of i=0
 Values of i=1
 Values of i=2
 Values of i=3
 Values of i=4
 Values of i=5

Thread Synchronization:

- ✓ Synchronization in Java is the capability to control the access of multiple threads to any shared resource.
- ✓ We can't predict the execution of threads if more threads are ready for execution
- ✓ If the thread is executed under the synchronization, no other thread will be interrupt until the thread completes its execution i.e., other threads will be in waiting stage

Ex: t1.start(); t2.start(); t3.start();



Note: t2(), t3() will wait until t1() completes

Program:

```

class ThreadSync implements Runnable
{
    static int count;
    public synchronized void run()
    {
        for(int i=1;i<=10000;i++)
        {
            count++;
        }
    }
}

public class ThreadSyncDemo {

    public static void main(String[] args) {
        ThreadSync ob = new ThreadSync();

        Thread t1 = new Thread(ob);
        Thread t2 = new Thread(ob);

        t1.start();
        t2.start();
    }
}
  
```

```

        try
        {
            t1.join();
            t2.join();
        }
        catch (Exception e)
        {
            System.out.println(e);
        }
        System.out.println(count);
    }
}
}

```

Output:

20000

Inter Thread Communication:

- ✓ Two or more threads can communicate with each other by using **wait()**, **notify()** and **notifyAll()** methods and these methods are available in java.lang package
- ✓ These 3 methods present in **Object class** because thread can call these methods on any java object
- ✓ Only synchronized threads can communicate with each other
- ✓ Inter - thread communication is a mechanism in which a thread goes into wait state until another thread sends a notification
- ✓ To call any of the above method on any object, thread should be owner of that object i.e., the thread should have lock of that object means the thread should be inside synchronized area otherwise we will get runtime exception.
- ✓ If a thread calls wait() method on any object, it immediately releases the lock of that particular object and enter into waiting state
- ✓ If a thread calls **notify()** or **notifyAll()** method on any object, it releases the lock of that object but may not immediately
- ✓ Except these 3 methods, there is no other method, where thread releases the lock

Example Program:

```

class Customer
{
    int amount = 10000;
    public synchronized void withdraw(int amount)
    {
        System.out.println("Going to withdraw");
        if(this.amount < amount)
        {
            System.out.println("Less balance waiting to deposit");
            try
            {
                wait();
            }
        }
    }
}

```

```
        }  
        catch (Exception e)  
        {  
            System.out.println(e);  
        }  
    }  
    this.amount = this.amount - amount;  
    System.out.println("Withdraw Completed");  
}  
public synchronized void deposit(int amount)  
{  
    System.out.println("Going to deposit");  
    this.amount = this.amount + amount;  
    System.out.println("Deposit Completed");  
    notify();  
}  
}  
public class InterThreadDemo {  
    public static void main(String[] args) {  
        Customer c = new Customer();  
        new Thread()  
        {  
            public void run()  
            {  
                c.withdraw(15000);  
            }  
        }.start();  
        new Thread()  
        {  
            public void run()  
            {  
                c.deposit(10000);  
            }  
        }.start();  
    }  
}
```

Output:

Going to withdraw
Less balance waiting to deposit
Going to deposit
Deposit Completed
Withdraw Completed

Thread priorities:

Each thread has a priority. Priorities are represented by a number between 1 and 10. In most cases, the thread scheduler schedules the threads according to their priority. But it is not guaranteed because it depends on JVM specification that which scheduling it chooses.

Note: The priority is given by the JVM or by the programmer itself explicitly.

There are **three constant variables** which are static and used to fetch priority of a Thread. They are as following:

1. public static int MIN_PRIORITY

It holds the minimum priority that can be given to a thread. The value for this is 1.

2. public static int NORM_PRIORITY

It is the default priority that is given to a thread if it is not defined. The value for this is 5.

3. public static int MAX_PRIORITY

It is the maximum priority that can be given to a thread. The value for this is 10.

Get and Set methods in Thread priority

1. public final int getPriority()

In Java, getPriority() method is in java.lang.Thread package. it is used to get the priority of a thread.

2. public final void setPriority(int newPriority)

In Java setPriority(int newPriority) method is in java.lang.Thread package. It is used to set the priority of a thread. The setPriority() method throws IllegalArgumentException if the value of new priority is above minimum and maximum limit.

Example Program:

```
class MyThread extends Thread
{
    public void run()
    {
        System.out.println("Thread Running...");
    }
    public static void main(String[] args)
    {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();
        t1.start();
        System.out.println("max thread priority = " + t1.MAX_PRIORITY);
        System.out.println("min thread priority = " + t1.MIN_PRIORITY);
        System.out.println("normal thread priority = " + t1.NORM_PRIORITY);

        System.out.println("priority of thread = " + t2.getPriority());
    }
}
```

```

        t2.setPriority(10);
        System.out.println("After change, priority of thread = " + t2.getPriority());
    }
}

```

Output:

Thread Running...
 max thread priority = 10
 min thread priority = 1
 normal thread priority = 5
 priority of thread = 5
 After change, priority of thread = 10

Multi Tasking Vs Multi Threading:

S. No	<u>Multi Tasking</u>	<u>Multi Threading</u>
1	Executing multiple tasks simultaneously where each task is a separate independent process (program)	Executing multiple tasks simultaneously where each task is a separate independent part of the same program
2	Sharing of resources (cpu, i/o, memory) among multiple processes	Sharing of resources among multiple threads of the same program
3	CPU switches multiple processes to complete its execution	CPU switches multiple threads of the same processes
4	Each program has its own address space	Multiple threads may share a common address space
5	Heavy weight process	Light weight process
6	Context switching is high cost	Context switching is low cost
7	Inter process communication cost is high	Inter thread communication cost is low
8	Not under control of OS and java	Under control of OS and java

Additional Programs:

Program: Multiple Threads creation using Thread Class

```
class Thread1 extends Thread{
    public void run()
    {
        try
        {
            Thread.sleep(1000);
        }
        catch (InterruptedException e)
        {
            System.out.println(e);
        }
        System.out.println("Good Morning");
    }
}
class Thread2 extends Thread{
    public void run()
    {
        try
        {
            Thread.sleep(2000);
        }
        catch (InterruptedException e)
        {
            System.out.println(e);
        }
        System.out.println("Hello");
    }
}
class Thread3 extends Thread{
    public void run(){
        try
        {
            Thread.sleep(3000);
        }
        catch (InterruptedException e)
        {
            System.out.println(e);
        }
        System.out.println("Welcome");
    }
}
public class ThreadDemo3 {
    public static void main(String[] args) {

        Thread1 t1 = new Thread1();
        Thread2 t2 = new Thread2();
        Thread3 t3 = new Thread3();

        t1.start();
```

```
        t2.start();
        t3.start();
    }
}
```

Output:

Good Morning
Hello
Welcome

Program: Multiple Threads creation using Runnable Interface

```
class Thread1 implements Runnable{
    public void run()
    {
        try
        {
            Thread.sleep(1000);
        }
        catch (InterruptedException e)
        {
            System.out.println(e);
        }
        System.out.println("Good Morning");
    }
}

class Thread2 implements Runnable{
    public void run()
    {
        try
        {
            Thread.sleep(2000);
        }
        catch (InterruptedException e)
        {
            System.out.println(e);
        }
        System.out.println("Hello");
    }
}

class Thread3 implements Runnable{
    public void run(){
        try
        {
            Thread.sleep(3000);
        }
        catch (InterruptedException e)
        {
            System.out.println(e);
        }
        System.out.println("Welcome");
    }
}
```

```
public class ThreadDemo3 {  
    public static void main(String[] args) {  
  
        Thread1 ob1 = new Thread1();  
        Thread2 ob2 = new Thread2();  
        Thread3 ob3 = new Thread3();  
  
        Thread t1 = new Thread(ob1);  
        Thread t2 = new Thread(ob2);  
        Thread t3 = new Thread(ob3);  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

Output:

Good Morning
Hello
Welcome